

The Gamer’s Path — Documentação Técnica Completa

Pedro Rivz

2026-05-01

Contents

1	Visão Geral do Projeto	4
2	Stack Tecnológica	4
2.1	Frontend	4
2.2	Backend	4
2.3	Infraestrutura	4
3	Arquitetura	5
3.1	Visão de Alto Nível	5
3.2	Estrutura de Diretórios	5
4	Banco de Dados	6
4.1	Schema SQLite	6
4.2	Configurações do SQLite	7
5	Variáveis de Ambiente	7
5.1	Servidor (server/.env)	7
6	API Reference	8
6.1	Autenticação	8
6.2	Endpoints	8
6.2.1	Auth — /api/auth	8
6.2.2	Jogos — /api/games	9
6.2.3	Biblioteca — /api/library	10
6.2.4	Backup — /api/backup	12
6.2.5	Health Check — /api/health	12
7	Integração Ollama	13
7.1	Fluxo de Extração	13
7.2	Configuração Docker	13

8	Sistema de Cache Redis	13
9	Frontend — Páginas	14
9.1	Home (/)	14
9.2	Search (/search)	14
9.3	GameDetail (/game/:id)	14
9.4	Library (/library)	14
9.5	Auth (/auth)	14
9.6	Profile (/)	15
10	Frontend — Componentes	15
10.1	Navigation	15
10.2	LevelTracker	15
10.3	GameCard	15
10.4	OptimizedImage	15
10.5	SkeletonCard	15
10.6	Toast	15
10.7	PrivateRoute	15
10.8	Confetti	16
11	Gerenciamento de Estado	16
11.1	useAuth (src/hooks/useAuth.ts)	16
11.2	useLibrary (src/store/library.ts)	16
12	O Problema dos Dois IDs	17
13	TypeScript — Interfaces Principais	17
13.1	Frontend (src/types/speedrun.ts)	17
13.2	Backend (server/types.ts)	18
14	Convenções de Desenvolvimento	19
14.1	TypeScript	19
14.2	Tailwind CSS v4	19
14.3	Backend	19
14.4	Animações	19
15	Docker e Deploy	19
15.1	docker-compose.yml	19
15.2	Comandos Make	20
15.3	Comandos de Dev	20

16 Problemas Conhecidos	21
17 Changelog	21
17.1 [Unreleased]	21
17.1.1 Fixed	21
17.1.2 Added	21
17.2 [1.1.0] — 2026-04-25	22
17.2.1 Added	22
17.2.2 Fixed	22
17.3 [1.0.0] — 2026-04-25	22
17.3.1 Added	22
18 Código Fonte Completo	22
18.1 server/server.ts	22
18.2 server/db.ts	24
18.3 server/auth.ts	25
18.4 server/middleware/auth.ts	25
18.5 server/routes/auth.ts	25
18.6 server/routes/games.ts	27
18.7 src/App.tsx	28
18.8 src/hooks/useAuth.ts	30
18.9 src/store/library.ts	31
18.10src/api/client.ts (seleção)	33
19 Guia de Início Rápido	34
19.1 Desenvolvimento Local	34
19.2 Deploy com Docker	35

1 Visão Geral do Projeto

The Gamer's Path (TGP) é uma aplicação web mobile-first para rastrear a conclusão de jogos single-player, capítulo por capítulo. Usuários buscam jogos via a API pública do Speedrun.com, adicionam-nos a uma biblioteca pessoal e marcam níveis/capítulos como concluídos.

- **Repositório:** github.com/pedrorivz/thegamerspath
- **Versão atual:** 1.1.0
- **Linguagem UI:** Português do Brasil
- **Implantação:** Self-hosted via Docker Compose

2 Stack Tecnológica

2.1 Frontend

Tecnologia	Versão	Finalidade
React	19.x	Framework de UI
TypeScript	~6.0	Tipagem estática
Vite	8.x	Bundler e dev server
Tailwind CSS	v4.x	Estilização utility-first
Framer Motion	12.x	Animações e transições
Zustand	5.x	Gerenciamento de estado global
React Router DOM	7.x	Roteamento SPA
Sonner	2.x	Notificações toast

2.2 Backend

Tecnologia	Versão	Finalidade
Node.js	24.x	Runtime
Express	4.x	Framework HTTP
TypeScript	5.x	Tipagem estática
better-sqlite3	11.x	Banco de dados SQLite
ioredis	5.x	Cliente Redis (opcional)
jsonwebtoken	9.x	Autenticação JWT
bcryptjs	2.x	Hash de senhas
cheerio	1.x	Web scraping (Ollama)

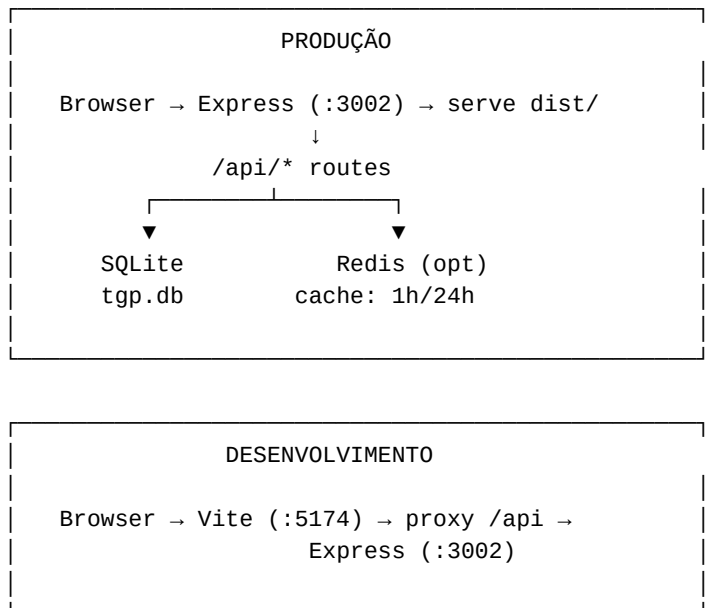
2.3 Infraestrutura

Componente	Descrição
Docker + Docker Compose	Containerização e orquestração
Redis 7 Alpine	Cache de busca e detalhes de jogos
SQLite (WAL mode)	Banco de dados principal persistido em volume

Componente	Descrição
Ollama (opcional)	LLM local para extração de capítulos

3 Arquitetura

3.1 Visão de Alto Nível



Em produção, frontend e backend rodam no mesmo processo Express (porta 3002). Em dev, o Vite em :5174 faz proxy de /api/* para o Express em :3002.

3.2 Estrutura de Diretórios

```

thegamerspath/
├── src/
│   ├── App.tsx           React SPA
│   │                   Roteamento, auth init, library sync
│   ├── api/
│   │   ├── client.ts     Todas as chamadas /api/* + backendToLibrary()
│   │   └── speedrun.ts    Helpers para API Speedrun.com
│   ├── hooks/
│   │   └── useAuth.ts     Store Zustand de autenticação
│   ├── store/
│   │   └── library.ts     Store Zustand da biblioteca
│   ├── types/
│   │   └── speedrun.ts    Interfaces TypeScript compartilhadas
│   ├── components/
│   │   ├── Navigation.tsx Bottom tab bar
│   │   └── PrivateRoute.tsx Guard de autenticação
  
```

			GameCard.tsx	Card de resultado de busca
			LevelTracker.tsx	Checklist de capítulos + progress bar
			Confetti.tsx	Canvas particle system
			Toast.tsx	Wrapper do Sonner
			SkeletonCard.tsx	Skeletons de carregamento
			OptimizedImage.tsx	Imagem lazy com fallback
			└─ pages/	
			Home.tsx	Dashboard com estatísticas
			Search.tsx	Busca com debounce 450ms
			GameDetail.tsx	Hero do jogo, tracker, notas
			Library.tsx	Lista completa da biblioteca
			Auth.tsx	Formulário login/registro
			Profile.tsx	Estatísticas + logout + backup
			└─ server/	
			server.ts	Setup Express, Redis, Ollama, static
			db.ts	Conexão SQLite, WAL, schema auto-create
			auth.ts	signToken / verifyToken (JWT)
			types.ts	Interfaces backend + Express augmentation
			ollama.ts	Cliente Ollama + extração de capítulos
			middleware/	
			auth.ts	requireAuth middleware
			└─ routes/	
			auth.ts	POST /register, POST /login, GET /me
			library.ts	CRUD biblioteca + Ollama + notas
			games.ts	Proxy Speedrun.com + cache Redis
			backup.ts	Export/import JSON backup
			└─ dist/	Build Vite (servido pelo Express em prod)
			└─ docker-compose.yml	
			└─ Dockerfile	
			└─ Makefile	
			└─ CLAUDE.md	Instruções para agentes AI

4 Banco de Dados

4.1 Schema SQLite

```
-- Usuários
CREATE TABLE IF NOT EXISTS users (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  email TEXT UNIQUE NOT NULL COLLATE NOCASE,
  password_hash TEXT NOT NULL,
  created_at TEXT NOT NULL DEFAULT (datetime('now'))
);

-- Jogos na biblioteca
CREATE TABLE IF NOT EXISTS library_games (
  id TEXT PRIMARY KEY,                -- UUID gerado pelo backend
  user_id INTEGER NOT NULL,
  speedrun_id TEXT NOT NULL,          -- ID na Speedrun.com API
  name TEXT NOT NULL,
```

```

cover_url TEXT,
abbreviation TEXT NOT NULL DEFAULT '',
released INTEGER NOT NULL DEFAULT 0,
platforms TEXT NOT NULL DEFAULT '[]', -- JSON string
genres TEXT NOT NULL DEFAULT '[]', -- JSON string
added_at TEXT NOT NULL DEFAULT (datetime('now')),
ollama_status TEXT, -- null | 'processing' | 'done' | 'failed'
FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);

-- Capítulos/níveis
CREATE TABLE IF NOT EXISTS library_levels (
  id TEXT PRIMARY KEY, -- UUID
  game_id TEXT NOT NULL,
  speedrun_level_id TEXT NOT NULL, -- ID na Speedrun.com (ou UUID para manuais)
  name TEXT NOT NULL,
  completed INTEGER NOT NULL DEFAULT 0, -- 0 = false, 1 = true
  completed_at TEXT,
  FOREIGN KEY (game_id) REFERENCES library_games(id) ON DELETE CASCADE
);

-- Notas do jogador
CREATE TABLE IF NOT EXISTS game_notes (
  id TEXT PRIMARY KEY, -- UUID
  game_id TEXT NOT NULL,
  content TEXT NOT NULL,
  created_at TEXT NOT NULL DEFAULT (datetime('now')),
  updated_at TEXT NOT NULL DEFAULT (datetime('now')),
  FOREIGN KEY (game_id) REFERENCES library_games(id) ON DELETE CASCADE
);

-- Índices
CREATE INDEX IF NOT EXISTS idx_library_games_user_id ON library_games(user_id);
CREATE INDEX IF NOT EXISTS idx_library_levels_game_id ON library_levels(game_id);
CREATE INDEX IF NOT EXISTS idx_game_notes_game_id ON game_notes(game_id);

```

4.2 Configurações do SQLite

- **WAL mode** (PRAGMA journal_mode = WAL) — melhor concorrência de leitura
- **Foreign keys** (PRAGMA foreign_keys = ON) — integridade referencial ativada
- **Migration segura** — ALTER TABLE ADD COLUMN ollama_status é executada no startup; ignorada se a coluna já existir

5 Variáveis de Ambiente

5.1 Servidor (server/.env)

Variável	Padrão	Obrigatório	Descrição
PORT	3001	Sim	Deve ser 3002 para o proxy Vite funcionar
JWT_SECRET	dev-secret-...	Sim em prod	Segredo para assinar JWT — mudar antes de qualquer deploy
REDIS_URL	—	Não	Ex: redis://localhost:6379; sem cache se ausente
DATA_DIR	server/data/	Não	Diretório do arquivo tgp.db
OLLAMA_URL	—	Não	Ex: http://localhost:11434; Ollama desativado se ausente
OLLAMA_MODEL	llama3.2	Não	Modelo Ollama a usar

Aviso: O arquivo server/.env não é commiteado. Copiar de server/.env.example:

```
cp server/.env.example server/.env
```

6 API Reference

6.1 Autenticação

Todos os endpoints protegidos exigem o header:

Authorization: Bearer <JWT_TOKEN>

O token tem validade de **7 dias**.

6.2 Endpoints

6.2.1 Auth — /api/auth

6.2.1.1 POST /api/auth/register Registra novo usuário.

Body:

```
{ "email": "user@example.com", "password": "minimo6chars" }
```

Response 201:

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXZWQ",
  "user": { "id": 1, "email": "user@example.com" }
}
```

Erros: 400 (campos inválidos), 409 (email já cadastrado)

6.2.1.2 POST /api/auth/login Autentica usuário existente.

Body:

```
{ "email": "user@example.com", "password": "senha" }
```

Response 200:

```
{
  "token": "eyJhbGciOi...",
  "user": { "id": 1, "email": "user@example.com" }
}
```

Erros: 400 (campos ausentes), 401 (credenciais inválidas)

6.2.1.3 GET /api/auth/me Valida o token e retorna o usuário atual.

Response 200:

```
{ "user": { "id": 1, "email": "user@example.com" } }
```

Erros: 401 (sem token ou token inválido)

6.2.2 Jogos — /api/games

6.2.2.1 GET /api/games/search?q={query} Busca jogos na Speedrun.com API com múltiplas variantes da query. Resultados cacheados no Redis por 1 hora.

Query params: q — nome do jogo

Response 200:

```
{
  "data": [
    {
      "id": "v1pxjz68",
      "names": { "international": "Super Mario World" },
      "abbreviation": "smw",
      "assets": { "cover-large": { "uri": "https://..." } },
      "platforms": { "data": [{ "id": "...", "name": "SNES" }] },
      "genres": { "data": [{ "id": "...", "name": "Platformer" }] },
      "levels": { "data": [{ "id": "...", "name": "Yoshi's Island 1" }] }
    }
  ]
}
```

Header X-Cache: HIT | MISS indica se o resultado veio do Redis.

6.2.2.2 GET /api/games/:id Busca detalhes de um jogo pelo ID da Speedrun.com. Cacheado por 24 horas.

Erros: 404 (jogo não encontrado), 502 (erro ao contatar Speedrun.com)

6.2.3 Biblioteca — /api/library

Todos os endpoints requerem autenticação. Operações de escrita verificam posse via `WHERE id = ? AND user_id = ?`.

6.2.3.1 GET /api/library Lista todos os jogos da biblioteca do usuário com capítulos e notas.

Response 200:

```
{
  "data": [
    {
      "id": "uuid-backend",
      "speedrunId": "v1pxjz68",
      "name": "Super Mario World",
      "coverUrl": "https://...",
      "abbreviation": "smw",
      "released": 1990,
      "platforms": ["SNES"],
      "genres": ["Platformer"],
      "levels": [
        { "id": "uuid", "name": "Yoshi's Island 1", "completed": false }
      ],
      "notes": [
        { "id": "uuid", "content": "Texto...", "createdAt": "...", "updatedAt": "..." }
      ],
      "ollamaStatus": null,
      "addedAt": "2026-04-01T..."
    }
  ]
}
```

6.2.3.2 POST /api/library Adiciona jogo à biblioteca.

Body:

```
{
  "speedrun_id": "v1pxjz68",
  "name": "Super Mario World",
  "cover_url": "https://...",
  "abbreviation": "smw",
  "released": 1990,
  "platforms": ["SNES"],
  "genres": ["Platformer"],
  "levels": [{ "id": "speedrun-level-id", "name": "Yoshi's Island 1" }],
  "useOllama": false
}
```

Se useOllama: true e levels: [], a extração Ollama é iniciada em background e ollamaStatus fica "processing".

Response 201: Objeto BackendGame do jogo adicionado.

Erros: 400 (campos obrigatórios ausentes), 409 (jogo já na biblioteca)

6.2.3.3 DELETE /api/library/:gameId Remove jogo da biblioteca. Cascata deleta capítulos e notas.

Response 204 (sem body)

6.2.3.4 PATCH /api/library/:gameId/level Alterna o estado completed/incompleto de um capítulo.

Body: { "levelId": "uuid" }

Response 200:

```
{ "data": { "id": "uuid", "name": "Cap 1", "completed": true, "completedAt": "..." } }
```

6.2.3.5 POST /api/library/:gameId/levels Adiciona um capítulo manualmente.

Body: { "name": "Capítulo X" } (máx 100 chars)

Response 201: Objeto do nível criado.

6.2.3.6 POST /api/library/:gameId/levels/bulk Adiciona múltiplos capítulos de uma vez.

Body: { "names": ["Cap 1", "Cap 2", "Cap 3"] }

Response 201: Array de níveis criados.

6.2.3.7 POST /api/library/:gameId/notes Adiciona nota a um jogo.

Body: { "content": "Texto da nota" }

Response 201: Objeto da nota criada.

6.2.3.8 PATCH /api/library/:gameId/notes/:noteId Edita conteúdo de uma nota existente.

Body: { "content": "Novo texto" }

Response 200: Objeto da nota atualizada.

6.2.3.9 DELETE /api/library/:gameId/notes/:noteId Remove uma nota.

Response 204 (sem body)

6.2.3.10 GET /api/library/:gameId/status Consulta o status do processamento Ollama para um jogo.

Response 200:

```
{ "data": { "ollamaStatus": "processing", "levelsCount": 0 } }
```

6.2.3.11 POST /api/library/:gameId/ollama Inicia (ou reinicia) a extração de capítulos via Ollama para um jogo existente na biblioteca.

Response 202: { "data": { "ollamaStatus": "processing" } }

Erros: 503 (Ollama não disponível), 404 (jogo não encontrado), 409 (extração já em andamento)

6.2.4 Backup — /api/backup

6.2.4.1 GET /api/backup/export Exporta toda a biblioteca do usuário como arquivo JSON.

Response 200: Download de arquivo `tgp-backup-YYYY-MM-DD.json`

Formato:

```
{
  "version": 1,
  "exportedAt": "2026-05-01T...",
  "games": [
    {
      "speedrunId": "v1pxjz68",
      "name": "Super Mario World",
      "coverUrl": "...",
      "abbreviation": "smw",
      "released": 1990,
      "platforms": ["SNES"],
      "genres": ["Platformer"],
      "addedAt": "...",
      "levels": [
        { "name": "Yoshi's Island 1", "speedrunLevelId": "...", "completed": true, "completedAt": "..." },
      ],
      "notes": [
        { "content": "...", "createdAt": "...", "updatedAt": "..." }
      ]
    }
  ]
}
```

6.2.4.2 POST /api/backup/import Importa um backup JSON. **Substitui** toda a biblioteca atual do usuário.

Body: Objeto `BackupFile` (formato acima)

Response 200: { "data": { "imported": 5 } }

6.2.5 Health Check — /api/health

6.2.5.1 GET /api/health Verifica o status do servidor.

Response 200:

```
{
  "status": "ok",
  "redis": true,
  "ollama": false,
  "timestamp": "2026-05-01T..."
}
```

7 Integração Ollama

O Ollama é um runtime de LLM local opcional que permite extrair automaticamente a lista de capítulos de um jogo quando o Speedrun.com não fornece níveis.

7.1 Fluxo de Extração

1. O usuário adiciona um jogo com `levels: []` e `useOllama: true`, **ou** clica em “Buscar capítulos com Ollama” no `GameDetail`
2. O backend seta `ollama_status = 'processing'` e inicia `processOllamaChapters()` em background (fire-and-forget)
3. `extractChapters()` em `ollama.ts`:
 - a. Busca URLs de walkthrough no DuckDuckGo (query: `{gameName} walkthrough chapters guide`)
 - b. Extrai headings H1/H2/H3 e itens de listas de até 3 páginas via Cheerio
 - c. Envia contexto ao Ollama via `/api/chat` com system prompt pedindo JSON `{"chapters": [...]}`
 - d. Parseia e valida a resposta
4. O backend insere os capítulos na tabela `library_levels`
5. Seta `ollama_status = 'done'` (ou `'failed'` se nenhum capítulo foi extraído)
6. O frontend faz polling em `GET /api/library/:gameId/status` a cada 5s durante o processamento
7. Ao detectar `'done'`, sincroniza a biblioteca (`sync()`)

7.2 Configuração Docker

Para usar Ollama via Docker Compose, descomentar no `docker-compose.yml`:

```
environment:
  OLLAMA_URL: http://host.docker.internal:11434
  OLLAMA_MODEL: llama3.1:8b
extra_hosts:
  - "host.docker.internal:host-gateway"
```

8 Sistema de Cache Redis

O Redis é completamente opcional — o app funciona normalmente sem ele.

Cache Key	TTL	Conteúdo
search:{query}	3600s (1h)	Resultados de busca normalizados
game:{speedrunId}	86400s (24h)	Detalhes de um jogo (embeds incluídos)

Política de degradação: Se o Redis não estiver disponível, as requisições são respondidas normalmente sem cache. O header X-Cache: HIT/MISS indica o comportamento.

9 Frontend — Páginas

9.1 Home (/)

Dashboard com estatísticas do usuário: - Total de jogos na biblioteca - Total de capítulos completados vs. total - Percentual geral de conclusão - 3 jogos adicionados mais recentemente com suas barras de progresso

9.2 Search (/search)

- Campo de busca com debounce de 450ms
- Variantes da query testadas em paralelo (normal, _, sem espaço, abreviação)
- Skeleton cards durante carregamento
- Badge “Na biblioteca” em jogos já adicionados
- Tap no card navega para /game/:speedrunId

9.3 GameDetail (/game/:id)

- Carrega dados da Speedrun.com API via backend proxy
- **Para jogos na biblioteca:** exibe cover, nome, plataforma, progresso e tracker
- **Para jogos não na biblioteca:** exibe detalhes e botão “Adicionar”
- Botão “Buscar capítulos com Ollama” (visível quando: jogo na biblioteca, sem capítulos, Ollama online)
- Banner de processamento com spinner durante extração Ollama
- LevelTracker com toggle de completion, barra de progresso, confetti e haptics
- Seção de notas com add/edit/delete inline
- Sheet de confirmação para remover jogo

9.4 Library (/library)

- Lista completa dos jogos da biblioteca do usuário
- Progresso por jogo (X/Y capítulos)
- Links usam game.speedrunId (não game.id) para navegar corretamente
- Animação de lista com Framer Motion

9.5 Auth (/auth)

- Formulário que alterna entre login e registro
- Validação: email com regex, senha mínimo 6 chars
- Toast de erro em falha, redirecionamento automático em sucesso

9.6 Profile (/)

- Estatísticas do usuário (jogos, capítulos, % conclusão)
 - Toggle para ativar/desativar busca Ollama (persiste em localStorage)
 - Botão de exportar backup (download JSON)
 - Botão de importar backup (upload arquivo, sheet de confirmação)
 - Botão de logout
-

10 Frontend — Componentes

10.1 Navigation

Bottom tab bar com ícones para Home, Search, Library e Profile. Ocultado na rota /auth.

10.2 LevelTracker

Checklist de capítulos com: - Toggle de completed (otimista via Zustand) - Barra de progresso animada - Confetti (canvas 80 partículas, 2 bursts) ao completar todos os capítulos - Haptic feedback via `navigator.vibrate()` - Formulário inline para adicionar capítulo (modo simples e bulk)

10.3 GameCard

Card de resultado de busca com: - Imagem de capa otimizada - Nome do jogo e plataformas - Badge “Na biblioteca” se o jogo já foi adicionado

10.4 OptimizedImage

Imagem lazy com: - Placeholder skeleton durante carregamento - Fallback visual em caso de erro

10.5 SkeletonCard

Três variantes de shimmer placeholder: - card — resultado de busca - detail-hero — hero da página de detalhes - level-row — linha de capítulo

10.6 Toast

Wrapper do Sonner com métodos tipados: - `toast.success()`, `toast.error()`, `toast.info()`, `toast.trophy()`

10.7 PrivateRoute

Guard de autenticação que aguarda `initialized` antes de redirecionar para /auth.

10.8 Confetti

Canvas particle system autônomo: - 80 partículas com velocidade, cor e ângulo aleatórios - Dois bursts com delay de 700ms - Cleanup automático do canvas após animação

11 Gerenciamento de Estado

11.1 useAuth (src/hooks/useAuth.ts)

Store Zustand para autenticação:

Campo/Método	Tipo	Descrição
token	string \ null	JWT atual (lido do localStorage na criação)
user	AuthUser \ null	Usuário autenticado
loading	boolean	Flag de operação em andamento
initialized	boolean	true após init() completar
init()	async	Valida token no backend; limpa se inválido
login()	async	Login + salva token no localStorage
register()	async	Registro + salva token
logout()	sync	Remove token do localStorage, limpa state

Importante: O store **não usa persist middleware** — o token é persistido manualmente no localStorage, o estado de usuário vem do backend.

11.2 useLibrary (src/store/library.ts)

Store Zustand para a biblioteca de jogos:

Campo/Método	Tipo	Descrição
games	LibraryGame[]	Lista de jogos na biblioteca
syncing	boolean	Flag de sincronização em andamento
sync()	async	Fetch completo da biblioteca do backend
addGame()	async	Adiciona jogo (backend + state)
removeGame()	async	Remove jogo com rollback otimista
toggleLevel()	async	Toggle completion otimista com rollback
addLevel()	async	Adiciona capítulo manual
addLevels()	async	Adiciona múltiplos capítulos (bulk)
addNote()	async	Adiciona nota
updateNote()	async	Edita nota
deleteNote()	async	Remove nota com rollback
setOllamaStatus()	sync	Atualiza status Ollama sem backend call
clear()	sync	Limpa state (chamado no logout)
hasGame(id)	sync	Verifica por speedrunId OR id

Campo/Método	Tipo	Descrição
getGame(id)	sync	Busca por speedrunId OR id

12 O Problema dos Dois IDs

LibraryGame tem dois campos de ID:

- **id** — UUID gerado pelo backend (chave primária no SQLite, usado para todas as chamadas de API da biblioteca)
- **speedrunId** — ID do jogo na Speedrun.com (usado para buscar dados da API Speedrun.com)

Regra de navegação:

Contexto	ID a usar no link /game/:id
Página de Busca	game.id (objeto SpeedrunGame)
Páginas de Biblioteca/Home	game.speedrunId (objeto LibraryGame)

Passar um UUID para a API do Speedrun.com retorna 404. Este foi um bug histórico no projeto.

13 TypeScript — Interfaces Principais

13.1 Frontend (src/types/speedrun.ts)

```
// Representa um jogo na biblioteca do usuário
interface LibraryGame {
  id: string;           // UUID backend
  speedrunId: string;   // ID Speedrun.com
  name: string;
  coverUrl: string | null;
  abbreviation: string;
  released: number;
  platforms: string[];
  genres: string[];
  levels: LibraryLevel[];
  notes: LibraryNote[];
  ollamaStatus: string | null;
  addedAt: number;      // timestamp em ms
}

interface LibraryLevel {
  id: string;
  name: string;
  completed: boolean;
}
```

```

interface LibraryNote {
  id: string;
  content: string;
  createdAt: string;
  updatedAt: string;
}

// Representa um jogo da API Speedrun.com
interface SpeedrunGame {
  id: string;
  names: { international: string; japanese?: string; twitch?: string };
  abbreviation: string;
  weblink: string;
  released: number;
  assets: SpeedrunAssets; // contém cover-tiny, cover-small, cover-medium, cover-large, etc.
  platforms: string[] | { data: SpeedrunPlatform[] };
  genres: string[] | { data: SpeedrunGenre[] };
  levels?: { data: SpeedrunLevel[] };
}

```

13.2 Backend (server/types.ts)

```

// Linha da tabela library_games (snake_case, como no SQLite)
interface LibraryGame {
  id: string;
  user_id: number;
  speedrun_id: string;
  name: string;
  cover_url: string | null;
  abbreviation: string;
  released: number;
  platforms: string; // JSON string de string[]
  genres: string; // JSON string de string[]
  added_at: string;
  ollama_status: string | null;
}

// Body do POST /api/library
interface AddGameBody {
  speedrun_id: string;
  name: string;
  cover_url?: string | null;
  abbreviation?: string;
  released?: number;
  platforms?: string[];
  genres?: string[];
  levels: { id: string; name: string }[];
  useOllama?: boolean;
}

```

14 Convenções de Desenvolvimento

14.1 TypeScript

- `erasableSyntaxOnly: true` no frontend — **proibido** usar `parameter properties` em construtores:

```
// ERRADO
constructor(public x: number) {}
// CORRETO
x: number;
constructor(x: number) { this.x = x; }
```

- `noUnusedLocals` e `noUnusedParameters` — parâmetros intencionalmente não usados devem ter prefixo `_`
- `verbatimModuleSyntax` no frontend — sempre `import type` para imports de tipo

14.2 Tailwind CSS v4

- Plugin `@tailwindcss/vite` — **nunca criar `tailwind.config.js`** (ignorado silenciosamente no v4)
- Toda configuração de tema em `@theme directives` no `src/index.css`
- Cores hardcoded em hex no JSX: `#0f0f1a`, `#1a1a2e`, `#7c3aed`

14.3 Backend

- Todas as chamadas ao DB usam `better-sqlite3` de forma **síncrona** — sem `async/await` em handlers de rota para operações DB
- Propriedade de recursos verificada com `WHERE id = ? AND user_id = ?` em toda operação mutante
- `platforms` e `genres` armazenados como JSON strings no SQLite

14.4 Animações

- Todas as animações usam `Framer Motion`
 - `AnimatePresence` obrigatório para animações de saída
 - Transições de página pelo `key={location.pathname}` no `Suspense` do `App.tsx`
 - **Não usar** `CSS transition` em propriedades já animadas pelo `Framer Motion`
-

15 Docker e Deploy

15.1 docker-compose.yml

```
services:
  web:
    build: .
    ports:
      - "3002:3002"
    environment:
      NODE_ENV: production
```

```

    PORT: 3002
    REDIS_URL: redis://redis:6379
    DATA_DIR: /app/server/data
    JWT_SECRET: ${JWT_SECRET:-change-this-secret-before-production}
    # Opcional para Ollama:
    # OLLAMA_URL: http://host.docker.internal:11434
    # OLLAMA_MODEL: llama3.1:8b
volumes:
  - sqlite-data:/app/server/data
depends_on:
  redis:
    condition: service_healthy
restart: unless-stopped
healthcheck:
  test: ["CMD", "wget", "-qO-", "http://localhost:3002/api/health"]
  interval: 30s

redis:
  image: redis:7-alpine
  command: >
    redis-server --appendonly yes
    --maxmemory 128mb --maxmemory-policy allkeys-lru
  volumes:
    - redis-data:/data
  restart: unless-stopped

volumes:
  sqlite-data:
  redis-data:

```

15.2 Comandos Make

```

make up           # docker compose up -d
make down         # docker compose down
make logs         # docker compose logs -f
make restart      # reinicia apenas o container web
make build        # docker compose build
make clean        # docker compose down -v + rm tgp.db

```

15.3 Comandos de Dev

```

# Iniciar tudo (frontend :5174, backend :3002)
npm run dev:all

# Apenas frontend
npm run dev

# Apenas backend
npm run dev:server # ou: cd server && npm run dev

# Type check
npx tsc --noEmit           # frontend

```

```
cd server && npx tsc --noEmit # backend
```

```
# Build de produção
```

```
npm run build
```

16 Problemas Conhecidos

1. **N+1 query em GET /api/library** — uma query por jogo para buscar capítulos e notas. Aceitável na escala atual.
 2. **Sem rate limiting nos endpoints de auth** — /register e /login não têm express-rate-limit. Não adicionar fluxos de auth sem também adicionar rate limiting.
 3. **Logout não invalida JWT no servidor** — tokens permanecem válidos por 7 dias após logout. Não há token blacklist.
 4. **SkeletonLevel usa Math.random() inline** — larguras não determinísticas por render. Inofensivo mas tecnicamente impuro.
 5. **Ollama faz web scraping sem respeitar robots.txt** — aceito para uso pessoal/self-hosted.
-

17 Changelog

17.1 [Unreleased]

17.1.1 Fixed

- Página de detalhes agora exibe dados da biblioteca (nome, cover, tracker) quando o ID Speedrun.com não é encontrado na API
- Links da biblioteca e home page agora usam speedrunId em vez do UUID backend
- Dockerfile agora copia server/ollama.ts, corrigindo crash no startup do container

17.1.2 Added

- Botão manual de importação Ollama na página de detalhes para jogos sem capítulos
- Integração Ollama para extração automática de capítulos via LLM local
- Toggle de configuração Ollama na página de Perfil (persiste em localStorage)
- Banner de processamento durante extração Ollama com polling a cada 5s
- GET /api/library/:gameId/status — endpoint para consultar progresso da extração
- Variáveis OLLAMA_URL e OLLAMA_MODEL
- Coluna ollama_status na tabela library_games

17.2 [1.1.0] — 2026-04-25

17.2.1 Added

- Entrada manual de capítulos via formulário inline
- Importação bulk de capítulos (colar lista, um por linha)
- Notas/diário por jogo com suporte a create, edit e delete
- Exportar backup em JSON pela página de Perfil
- Importar backup com sheet de confirmação

17.2.2 Fixed

- Links de Library e Home agora usam o ID correto (Speedrun.com), não o UUID backend

17.3 [1.0.0] — 2026-04-25

17.3.1 Added

- Busca de jogos via Speedrun.com API
 - Biblioteca pessoal com cover art, plataforma, gênero e lista de capítulos
 - Tracker de capítulos com barra de progresso animada
 - Confetti + vibração háptica ao completar jogo
 - Notificações toast
 - Skeleton loaders
 - Animações Framer Motion
 - Imagens otimizadas com blur-up placeholder
 - Autenticação JWT (registro/login com biblioteca isolada por usuário)
 - Armazenamento SQLite com WAL mode
 - Cache Redis opcional com degradação graceful
 - Backend Express com rotas protegidas
 - Lazy loading de páginas React
 - Suporte Docker com multi-stage Dockerfile
 - Makefile para operações comuns
-

18 Código Fonte Completo

18.1 server/server.ts

```
import 'dotenv/config';
import express from 'express';
import cors from 'cors';
import path from 'path';
import Redis from 'ioredis';
import authRouter from './routes/auth';
import { createLibraryRouter } from './routes/library';
import { createGamesRouter } from './routes/games';
import backupRouter from './routes/backup';
import { initOllama } from './ollama';
```

```

import type { OllamaClient } from './ollama';

const app = express();
const PORT = parseInt(process.env.PORT || '3001', 10);
const REDIS_URL = process.env.REDIS_URL;
const OLLAMA_URL = process.env.OLLAMA_URL;
const OLLAMA_MODEL = process.env.OLLAMA_MODEL || 'llama3.2';
const IS_PROD = process.env.NODE_ENV === 'production';

let redis: Redis | null = null;
if (REDIS_URL) {
  redis = new Redis(REDIS_URL, { lazyConnect: true, maxRetriesPerRequest: 1 });
  redis.connect().catch(() => {
    console.warn('[Redis] Não foi possível conectar. Continuando sem cache.');
```

```

    redis = null;
  });
}

let ollama: OllamaClient | null = null;
if (OLLAMA_URL) {
  initOllama(OLLAMA_URL, OLLAMA_MODEL).then(client => {
    if (client) ollama = client;
  });
}

app.use(cors({
  origin: IS_PROD ? false : ['http://localhost:5173', 'http://localhost:5174'],
  credentials: true,
}));
app.use(express.json());

app.use('/api/auth', authRouter);
app.use('/api/library', createLibraryRouter(() => ollama));
app.use('/api/games', createGamesRouter(redis));
app.use('/api/backup', backupRouter);

app.get('/api/health', (_req, res) => {
  res.json({ status: 'ok', redis: redis !== null, ollama: ollama !== null, timestamp: new Date().toISOString() });
});

if (IS_PROD) {
  const DIST_PATH = path.join(__dirname, '../dist');
  app.use(express.static(DIST_PATH));
  app.get('*', (_req, res) => res.sendFile(path.join(DIST_PATH, 'index.html')));
}

app.listen(PORT, () => {
  console.log(`The Gamer's Path server em http://localhost:${PORT}`);
});

```

18.2 server/db.ts

```
import Database from 'better-sqlite3';
import path from 'path';
import fs from 'fs';

const DATA_DIR = process.env.DATA_DIR || path.join(__dirname, 'data');
const DB_PATH = path.join(DATA_DIR, 'tgp.db');

if (!fs.existsSync(DATA_DIR)) fs.mkdirSync(DATA_DIR, { recursive: true });

const db = new Database(DB_PATH);
db.pragma('journal_mode = WAL');
db.pragma('foreign_keys = ON');

db.exec(`
CREATE TABLE IF NOT EXISTS users (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  email TEXT UNIQUE NOT NULL COLLATE NOCASE,
  password_hash TEXT NOT NULL,
  created_at TEXT NOT NULL DEFAULT (datetime('now'))
);
CREATE TABLE IF NOT EXISTS library_games (
  id TEXT PRIMARY KEY,
  user_id INTEGER NOT NULL,
  speedrun_id TEXT NOT NULL,
  name TEXT NOT NULL,
  cover_url TEXT,
  abbreviation TEXT NOT NULL DEFAULT '',
  released INTEGER NOT NULL DEFAULT 0,
  platforms TEXT NOT NULL DEFAULT '[]',
  genres TEXT NOT NULL DEFAULT '[]',
  added_at TEXT NOT NULL DEFAULT (datetime('now')),
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);
CREATE TABLE IF NOT EXISTS library_levels (
  id TEXT PRIMARY KEY,
  game_id TEXT NOT NULL,
  speedrun_level_id TEXT NOT NULL,
  name TEXT NOT NULL,
  completed INTEGER NOT NULL DEFAULT 0,
  completed_at TEXT,
  FOREIGN KEY (game_id) REFERENCES library_games(id) ON DELETE CASCADE
);
CREATE TABLE IF NOT EXISTS game_notes (
  id TEXT PRIMARY KEY,
  game_id TEXT NOT NULL,
  content TEXT NOT NULL,
  created_at TEXT NOT NULL DEFAULT (datetime('now')),
  updated_at TEXT NOT NULL DEFAULT (datetime('now')),
  FOREIGN KEY (game_id) REFERENCES library_games(id) ON DELETE CASCADE
);
CREATE INDEX IF NOT EXISTS idx_library_games_user_id ON library_games(user_id);
CREATE INDEX IF NOT EXISTS idx_library_levels_game_id ON library_levels(game_id);
```



```

    CREATE INDEX IF NOT EXISTS idx_game_notes_game_id ON game_notes(game_id);
  `);

  try {
    db.exec('ALTER TABLE library_games ADD COLUMN ollama_status TEXT');
  } catch { /* column already exists */ }

  export default db;

```

18.3 server/auth.ts

```

import jwt from 'jsonwebtoken';
import type { TokenPayload } from './types';

const JWT_SECRET = process.env.JWT_SECRET || 'dev-secret-change-in-production';
const JWT_EXPIRES_IN = '7d';

export function signToken(payload: TokenPayload): string {
  return jwt.sign(payload, JWT_SECRET, { expiresIn: JWT_EXPIRES_IN });
}

export function verifyToken(token: string): TokenPayload {
  return jwt.verify(token, JWT_SECRET) as TokenPayload;
}

```

18.4 server/middleware/auth.ts

```

import type { Request, Response, NextFunction } from 'express';
import { verifyToken } from '../auth';

export function requireAuth(req: Request, res: Response, next: NextFunction): void {
  const authHeader = req.headers.authorization;
  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    res.status(401).json({ error: 'Token de autenticação necessário' });
    return;
  }
  const token = authHeader.slice(7);
  try {
    req.user = verifyToken(token);
    next();
  } catch {
    res.status(401).json({ error: 'Token inválido ou expirado' });
  }
}

```

18.5 server/routes/auth.ts

```

import { Router } from 'express';
import bcrypt from 'bcryptjs';
import db from '../db';
import { signToken, verifyToken } from '../auth';

```

```

import type { User, AuthRequest } from '../types';

const router = Router();

router.post('/register', (req, res) => {
  const { email, password } = req.body as AuthRequest;

  if (!email || !password) {
    res.status(400).json({ error: 'Email e senha são obrigatórios' });
    return;
  }

  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  if (!emailRegex.test(email)) {
    res.status(400).json({ error: 'Email inválido' });
    return;
  }

  if (password.length < 6) {
    res.status(400).json({ error: 'Senha deve ter no mínimo 6 caracteres' });
    return;
  }

  const exists = db.prepare('SELECT id FROM users WHERE email = ?').get(email.toLowerCase());
  if (exists) {
    res.status(409).json({ error: 'Email já cadastrado' });
    return;
  }

  const hash = bcrypt.hashSync(password, 10);
  const result = db.prepare('INSERT INTO users (email, password_hash) VALUES (?, ?)').run(email.toLowerCase(), hash);
  const userId = result.lastInsertRowid as number;
  const token = signToken({ userId, email: email.toLowerCase() });
  res.status(201).json({ token, user: { id: userId, email: email.toLowerCase() } });
});

router.post('/login', (req, res) => {
  const { email, password } = req.body as AuthRequest;
  if (!email || !password) {
    res.status(400).json({ error: 'Email e senha são obrigatórios' });
    return;
  }
  const user = db.prepare('SELECT * FROM users WHERE email = ?').get(email.toLowerCase()) as User | undefined;
  if (!user || !bcrypt.compareSync(password, user.password_hash)) {
    res.status(401).json({ error: 'Email ou senha incorretos' });
    return;
  }
  const token = signToken({ userId: user.id, email: user.email });
  res.json({ token, user: { id: user.id, email: user.email } });
});

router.get('/me', (req, res) => {
  const authHeader = req.headers.authorization;
  if (!authHeader || !authHeader.startsWith('Bearer ')) {

```

```

    res.status(401).json({ error: 'Token necessário' });
    return;
  }
  try {
    const payload = verifyToken(authHeader.slice(7));
    res.json({ user: { id: payload.userId, email: payload.email } });
  } catch {
    res.status(401).json({ error: 'Token inválido' });
  }
});

export default router;

```

18.6 server/routes/games.ts

```

import { Router } from 'express';
import type { Redis } from 'ioredis';

const router = Router();
const SPEEDRUN_BASE = 'https://www.speedrun.com/api/v1';
const SEARCH_CACHE_TTL = 3600;
const GAME_CACHE_TTL = 86400;

function normalizeVariants(raw: string): string[] {
  const clean = raw.trim().toLowerCase();
  const variants = new Set<string>();
  variants.add(clean);
  variants.add(clean.replace(/s+/g, '_'));
  variants.add(clean.replace(/s+/g, ''));
  variants.add(clean.replace(/_/g, ' '));
  const abbr = clean.split(/[s_]+/).map((w: string) => w[0]).join('');
  if (abbr.length > 1) variants.add(abbr);
  return Array.from(variants);
}

export function createGamesRouter(redis: Redis | null) {
  router.get('/search', async (req, res) => {
    const query = (req.query.q as string || '').trim();
    if (!query) { res.json({ data: [] }); return; }

    const cacheKey = `search:${query.toLowerCase()}`;
    if (redis) {
      try {
        const cached = await redis.get(cacheKey);
        if (cached) { res.set('X-Cache', 'HIT'); res.json(JSON.parse(cached)); return; }
      } catch {}
    }

    const variants = normalizeVariants(query);
    const seen = new Set<string>();
    const results: unknown[] = [];

    await Promise.all(variants.map(async variant => {

```

```

    try {
      const url = `${SPEEDRUN_BASE}/games?name=${encodeURIComponent(variant)}&max=20&embed=levels,platforms,genres`;
      const response = await fetch(url);
      if (!response.ok) return;
      const json = await response.json() as { data: Array<{ id: string }> };
      for (const game of json.data) {
        if (!seen.has(game.id)) { seen.add(game.id); results.push(game); }
      }
    } catch {}
  }));

  const payload = { data: results };
  if (redis) {
    try { await redis.setex(cacheKey, SEARCH_CACHE_TTL, JSON.stringify(payload)); } catch {}
  }
  res.set('X-Cache', 'MISS');
  res.json(payload);
});

router.get('/:id', async (req, res) => {
  const { id } = req.params;
  const cacheKey = `game:${id}`;

  if (redis) {
    try {
      const cached = await redis.get(cacheKey);
      if (cached) { res.set('X-Cache', 'HIT'); res.json(JSON.parse(cached)); return; }
    } catch {}
  }

  try {
    const url = `${SPEEDRUN_BASE}/games/${id}?embed=levels,platforms,genres`;
    const response = await fetch(url);
    if (!response.ok) { res.status(response.status).json({ error: 'Jogo não encontrado' }); return; }
    const json = await response.json() as unknown;
    if (redis) {
      try { await redis.setex(cacheKey, GAME_CACHE_TTL, JSON.stringify(json)); } catch {}
    }
    res.set('X-Cache', 'MISS');
    res.json(json);
  } catch {
    res.status(502).json({ error: 'Erro ao contatar a Speedrun API' });
  }
});

return router;
}

```

18.7 src/App.tsx

```

import { lazy, Suspense, useEffect } from 'react';
import { BrowserRouter, Routes, Route, useLocation } from 'react-router-dom';
import { AnimatePresence } from 'framer-motion';

```

```

import { Navigation } from './components/Navigation';
import { PrivateRoute } from './components/PrivateRoute';
import { ToastProvider } from './components/Toast';
import { useAuth } from './hooks/useAuth';
import { useLibrary } from './store/library';

const Home = lazy(() => import('./pages/Home').then(m => ({ default: m.Home })));
const Search = lazy(() => import('./pages/Search').then(m => ({ default: m.Search })));
const GameDetail = lazy(() => import('./pages/GameDetail').then(m => ({ default: m.GameDetail })));
const Library = lazy(() => import('./pages/Library').then(m => ({ default: m.Library })));
const Auth = lazy(() => import('./pages/Auth').then(m => ({ default: m.Auth })));
const Profile = lazy(() => import('./pages/Profile').then(m => ({ default: m.Profile })));

const PageFallback = () => (
  <div className="min-h-dvh flex items-center justify-center">
    <div className="w-7 h-7 border-2 border-violet-700 border-t-violet-400 rounded-full animate-spin" />
  </div>
);

function AnimatedRoutes() {
  const location = useLocation();
  return (
    <AnimatePresence mode="wait" initial={false}>
      <Suspense fallback={<PageFallback />} key={location.pathname}>
        <Routes location={location}>
          <Route path="/auth" element={<Auth />} />
          <Route path="/" element={<PrivateRoute><Home /></PrivateRoute>} />
          <Route path="/search" element={<PrivateRoute><Search /></PrivateRoute>} />
          <Route path="/game/:id" element={<PrivateRoute><GameDetail /></PrivateRoute>} />
          <Route path="/library" element={<PrivateRoute><Library /></PrivateRoute>} />
          <Route path="/profile" element={<PrivateRoute><Profile /></PrivateRoute>} />
        </Routes>
      </Suspense>
    </AnimatePresence>
  );
}

function AppContent() {
  const { init, token } = useAuth();
  const sync = useLibrary(s => s.sync);
  useEffect(() => { init(); }, [init]);
  useEffect(() => { if (token) sync(); }, [token, sync]);
  return (
    <div className="max-w-lg mx-auto relative">
      <AnimatedRoutes />
      <Navigation />
    </div>
  );
}

export default function App() {
  return (
    <BrowserRouter>
      <AppContent />
    </BrowserRouter>
  );
}

```

```

    <ToastProvider />
  </BrowserRouter>
);
}

```

18.8 src/hooks/useAuth.ts

```

import { create } from 'zustand';
import * as api from '../api/client';

interface AuthState {
  token: string | null;
  user: api.AuthUser | null;
  loading: boolean;
  initialized: boolean;
  login: (email: string, password: string) => Promise<void>;
  register: (email: string, password: string) => Promise<void>;
  logout: () => void;
  init: () => Promise<void>;
}

export const useAuth = create<AuthState>((set, _get) => ({
  token: localStorage.getItem('tgp-token'),
  user: null,
  loading: false,
  initialized: false,

  init: async () => {
    const token = localStorage.getItem('tgp-token');
    if (!token) { set({ initialized: true }); return; }
    try {
      const { user } = await api.me();
      set({ token, user, initialized: true });
    } catch {
      localStorage.removeItem('tgp-token');
      set({ token: null, user: null, initialized: true });
    }
  },

  login: async (email, password) => {
    set({ loading: true });
    try {
      const result = await api.login(email, password);
      localStorage.setItem('tgp-token', result.token);
      set({ token: result.token, user: result.user, loading: false });
    } catch (err) { set({ loading: false }); throw err; }
  },

  register: async (email, password) => {
    set({ loading: true });
    try {
      const result = await api.register(email, password);
      localStorage.setItem('tgp-token', result.token);
    }
  },

```

```

        set({ token: result.token, user: result.user, loading: false });
    } catch (err) { set({ loading: false }); throw err; }
},

logout: () => {
    localStorage.removeItem('tgp-token');
    set({ token: null, user: null });
},
}));

```

18.9 src/store/library.ts

```

import { create } from 'zustand';
import * as api from '../api/client';
import type { LibraryGame, LibraryLevel, LibraryNote } from '../types/speedrun';

interface LibraryState {
    games: LibraryGame[];
    syncing: boolean;
    syncError: string | null;
    sync: () => Promise<void>;
    addGame: (payload: api.AddGamePayload) => Promise<void>;
    removeGame: (id: string) => Promise<void>;
    toggleLevel: (gameId: string, levelId: string) => Promise<void>;
    addLevel: (gameId: string, name: string) => Promise<void>;
    addLevels: (gameId: string, names: string[]) => Promise<void>;
    addNote: (gameId: string, content: string) => Promise<void>;
    updateNote: (gameId: string, noteId: string, content: string) => Promise<void>;
    deleteNote: (gameId: string, noteId: string) => Promise<void>;
    setOllamaStatus: (gameId: string, status: string) => void;
    clear: () => void;
    hasGame: (id: string) => boolean;
    getGame: (id: string) => LibraryGame | undefined;
}

export const useLibrary = create<LibraryState>((set, get) => ({
    games: [],
    syncing: false,
    syncError: null,

    sync: async () => {
        set({ syncing: true, syncError: null });
        try {
            const backendGames = await api.getLibrary();
            set({ games: backendGames.map(api.backendToLibrary), syncing: false });
        } catch (err) {
            set({ syncing: false, syncError: err instanceof Error ? err.message : 'Erro ao sincronizar' });
        }
    },

    addGame: async (payload) => {
        const bg = await api.addGame(payload);
        set(s => ({ games: [api.backendToLibrary(bg), ...s.games] }));
    }
}));

```

```

},

removeGame: async (id) => {
  set(s => ({ games: s.games.filter(g => g.id !== id) }));
  try { await api.removeGame(id); }
  catch (err) { get().sync(); throw err; }
},

toggleLevel: async (gameId, levelId) => {
  set(s => ({
    games: s.games.map(g => g.id !== gameId ? g : {
      ...g, levels: g.levels.map((l: LibraryLevel) =>
        l.id === levelId ? { ...l, completed: !l.completed } : l
      ),
    }),
  }));
  try { await api.toggleLevel(gameId, levelId); }
  catch {
    set(s => ({
      games: s.games.map(g => g.id !== gameId ? g : {
        ...g, levels: g.levels.map((l: LibraryLevel) =>
          l.id === levelId ? { ...l, completed: !l.completed } : l
        ),
      }),
    }));
  }
},

addLevel: async (gameId, name) => {
  const level = await api.addLevel(gameId, name);
  set(s => ({
    games: s.games.map(g => g.id !== gameId ? g : {
      ...g, levels: [...g.levels, { id: level.id, name: level.name, completed: false }]
    }),
  }));
},

addLevels: async (gameId, names) => {
  const levels = await api.addLevels(gameId, names);
  set(s => ({
    games: s.games.map(g => g.id !== gameId ? g : {
      ...g, levels: [...g.levels, ...levels.map(l => ({ id: l.id, name: l.name, completed: false }))]
    }),
  }));
},

addNote: async (gameId, content) => {
  const note = await api.addNote(gameId, content);
  set(s => ({
    games: s.games.map(g => g.id !== gameId ? g : {
      ...g, notes: [{ id: note.id, content: note.content, createdAt: note.createdAt, updatedAt: note.updatedAt }]
    }),
  }));
},

```



```

updateNote: async (gameId, noteId, content) => {
  const note = await api.updateNote(gameId, noteId, content);
  set(s => ({
    games: s.games.map(g => g.id !== gameId ? g : {
      ...g, notes: g.notes.map((n: LibraryNote) =>
        n.id === noteId ? { ...n, content: note.content, updatedAt: note.updatedAt } : n
      ),
    })),
  }));
},

deleteNote: async (gameId, noteId) => {
  set(s => ({
    games: s.games.map(g => g.id !== gameId ? g :
      { ...g, notes: g.notes.filter((n: LibraryNote) => n.id !== noteId) }
    ),
  }));
  try { await api.deleteNote(gameId, noteId); }
  catch { get().sync(); throw new Error('Não foi possível remover a nota'); }
},

setOllamaStatus: (gameId, status) => {
  set(s => ({ games: s.games.map(g => g.id === gameId ? { ...g, ollamaStatus: status } : g) }));
},

clear: () => set({ games: [], syncError: null }),

hasGame: (id) => {
  const { games } = get();
  return games.some(g => g.speedrunId === id || g.id === id);
},

getGame: (id) => {
  const { games } = get();
  return games.find(g => g.speedrunId === id || g.id === id);
},
}));

```

18.10 src/api/client.ts (seleção)

```

// Todas as chamadas HTTP ao /api/*
// JWT lido de localStorage.getItem('tgp-token')

class APIError extends Error {
  status: number;
  constructor(status: number, message: string) {
    super(message);
    this.status = status;
  }
}

async function request<T>(path: string, options: RequestInit = {}): Promise<T> {

```

```

const token = localStorage.getItem('tgp-token');
const headers: Record<string, string> = {
  'Content-Type': 'application/json',
  ...(options.headers as Record<string, string> ?? {}),
};
if (token) headers['Authorization'] = `Bearer ${token}`;
const res = await fetch(`/api${path}`, { ...options, headers });
if (res.status === 204) return undefined as T;
const body = await res.json();
if (!res.ok) throw new APIError(res.status, body.error ?? 'Erro desconhecido');
return body as T;
}

// Converte BackendGame (camelCase do servidor) para LibraryGame (shape do Zustand)
export function backendToLibrary(bg: BackendGame): LibraryGame {
  return {
    id: bg.id,
    speedrunId: bg.speedrunId,
    name: bg.name,
    coverUrl: bg.coverUrl,
    abbreviation: bg.abbreviation,
    released: bg.released,
    platforms: bg.platforms,
    genres: bg.genres,
    levels: bg.levels.map(l => ({ id: l.id, name: l.name, completed: l.completed })),
    notes: (bg.notes ?? []).map(n => ({ id: n.id, content: n.content, createdAt: n.createdAt, updatedAt:
    ollamaStatus: bg.ollamaStatus ?? null,
    addedAt: new Date(bg.addedAt).getTime(),
  });
}

```

19 Guia de Início Rápido

19.1 Desenvolvimento Local

```

# 1. Clonar repositório
git clone https://github.com/pedrorivz/thegamerspath.git
cd thegamerspath

# 2. Instalar dependências
npm install
cd server && npm install && cd ..

# 3. Configurar variáveis de ambiente
cp server/.env.example server/.env
# Editar server/.env: definir PORT=3002 e JWT_SECRET

# 4. Iniciar tudo
npm run dev:all
# Frontend: http://localhost:5174
# Backend: http://localhost:3002

```

19.2 Deploy com Docker

```
# 1. Configurar JWT_SECRET
export JWT_SECRET="sua-chave-super-secreta-aqui"

# 2. Subir containers
make up

# 3. Verificar saúde
curl http://localhost:3002/api/health

# 4. Ver logs
make logs
```

Documentação gerada em 2026-05-01 — versão 1.1.0